

B-Trees: History, Theory, and Implementation

From Disk-Optimized Indexes to Modern Database Engines

CMPT 225 — Data Structures and Programming

Final Project Report

Student Name: Raafay Riaz Qazi(301555443)

Abstract

B-trees are a family of self-balancing, multi-way search trees designed to minimize disk I/O by keeping data in wide, shallow structures. Originally proposed by Rudolf Bayer and Edward McCreight in 1972 to address the problem of maintaining large-ordered indexes on block-addressable storage, B-trees have since become a cornerstone of database and file-system design. This report surveys the B-tree from its foundational theory through its many practical variants—B⁺-trees, B*-trees, and cache-oblivious B-trees—to its modern role in production systems such as PostgreSQL, MySQL InnoDB, and NTFS.

We present the key algorithmic ideas underlying B-tree operations—search, insertion with node splitting, and deletion with key redistribution and node merging—alongside their asymptotic guarantees. We then trace the data structure's historical evolution, discuss how classic disk-oriented design decisions are being re-evaluated for SSD and in-memory hardware, and survey recent advancements including write-optimized variants such as the Bε-tree.

To ground the survey, we provide a complete C++ implementation of a B-tree of configurable minimum degree t . We validate correctness through five automated tests and evaluate performance with timing experiments across varying tree orders and dataset sizes up to 100,000 keys. Our benchmarks confirm the classical result that higher t dramatically reduces tree height and, consequently, the number of node accesses required per operation.

1. Introduction and Motivation

Modern computer systems exhibit a stark gap between the speed of main memory (nanoseconds) and the speed of persistent storage (milliseconds for spinning disk, microseconds for SSDs). Data structures designed for in-memory computation—such as binary search trees and hash tables—perform poorly when the dataset does not fit in RAM, because each pointer dereference that crosses a page boundary incurs a costly I/O operation. By the early 1970s, as commercial databases began managing record files far larger than available memory, this mismatch became a critical engineering problem.

The B-tree solves this problem by changing the branching factor from 2 (as in binary trees) to a large value m proportional to the block size of the underlying storage device. A node is sized to fill exactly one disk block, so loading a node costs exactly one I/O. With branching factor m , a B-tree of height h can store $O(m^h)$ keys, meaning a tree of height 3 with $m = 1000$ can index one billion keys with at most three disk reads per lookup. This logarithmic height in terms of I/O operations is the central insight that has made B-trees ubiquitous.

Beyond raw performance, B-trees maintain total ordering of keys, support efficient range queries, and guarantee worst-case $O(\log_B N)$ I/Os per operation (in the external-memory model), making them superior to hash tables for workloads requiring sorted access. These properties explain why B-trees—and their variant the B⁺-tree—are the default index structure in virtually every relational database engine in use today.

This report is organized as follows. Section 2 establishes background definitions. Section 3 presents the historical narrative of B-trees from 1972 to the present. Section 4 discusses recent advancements. Section 5 describes our C++ implementation. Section 6 presents experimental results. Section 7 discusses trade-offs and limitations, and Section 8 concludes.

2. Background and Definitions

2.1 Tree Terminology

Node: A unit of storage containing keys and, for internal nodes, pointers to child nodes. In the disk model, each node occupies exactly one disk block.

Degree / Order: The parameter t (minimum degree) governs node capacity. Each non-root node holds at least $t-1$ keys and at most $2t-1$ keys. Internal nodes hold one more child pointer than they hold keys. The branching factor ranges from t to $2t$.

Height: A B-tree storing N keys has height at most $\log_t[(N+1)/2]$. Because t is typically large, the height is small even for billions of keys.

Leaf vs. Internal Node: Leaf nodes hold actual data records (or record pointers). Internal nodes hold only keys and child pointers. In a B⁺-tree, all data lives in leaves, which are chained together for range scans.

2.2 The External-Memory Model

Analysis of B-tree algorithms uses the external-memory (EM) model introduced by Aggarwal and Vitter (1988) [1]. The model has three parameters: N (number of data items), B (block size—items per disk transfer), and M (memory size). The cost metric is the number of block transfers (I/Os). In this model, the optimal number of I/Os for sorting N items is $O((N/B) \log M/B(N/B))$, and B-trees achieve $O(\log_B N)$ I/Os per search or update.

2.3 B-Tree Invariants

A valid B-tree of minimum degree t satisfies:

- Every non-root node has at least $t-1$ keys.
- Every node has at most $2t-1$ keys (if a node reaches $2t-1$ keys, it is split).
- All leaves appear at the same depth (the tree is perfectly height-balanced).
- For a node with k keys, there are exactly $k+1$ children (internal nodes only).
- Keys within a node are stored in sorted order.
- The keys in $\text{child}[i]$ are all strictly between $\text{key}[i-1]$ and $\text{key}[i]$.

2.4 Operation Complexity

Operation	Time Complexity
Search	$O(t \log_t N)$
Insert	$O(t \log_t N)$
Delete	$O(t \log_t N)$
Range scan $[a,b]$	$O(t \log_t N + k)$

Table 1: B-tree operation complexities. k = number of results returned. B = block size.

3. Historical Narrative

3.1 Origins (1972): Bayer and McCreight

The B-tree was invented by Rudolf Bayer and Edward McCreight at Boeing Research Laboratories and published in *Acta Informatica* in 1972 [2]. Their paper, "Organization and Maintenance of Large Ordered Indexes," was motivated by the concrete problem of indexing large files on disk where the seek time dominated computation. Bayer and McCreight observed that binary search trees produced $O(\log_2 N)$ disk accesses per operation because each node holds a single key and two child pointers, fitting far fewer keys per block than the block capacity allowed.

Their key insight was to generalize the branching factor: if a single node occupies one disk block and can hold m keys, then the height of the tree—and thus the number of disk accesses—drops from $O(\log_2 N)$ to $O(\log_m N)$. They defined a family of balanced multiway trees with the invariants described in Section 2.3 and proved that all operations maintain balance in $O(\log_m N)$ disk accesses.

The original paper named the structure a "B-tree" without elaborating on what the "B" stands for. Knuth later speculated it may stand for "balanced," "broad," "bushy," or "Boeing" [3]. Regardless of etymology, the structure became foundational.

3.2 The B⁺-Tree Variant (1973–1979)

A practical limitation of the original B-tree for database applications is that data records are stored in internal nodes, consuming space that could otherwise be used for additional key pointers. The B⁺-tree (Knuth, 1973; Comer, 1979) [3, 4] addresses this by storing all records exclusively in leaf nodes and storing only separator keys in internal nodes. This change has two important effects:

- Internal nodes can hold more keys, increasing the branching factor and reducing height.
- Leaf nodes form a doubly-linked list, enabling efficient sequential access for range queries without traversing the tree.

Douglas Comer's 1979 ACM Computing Surveys article "The Ubiquitous B-Tree" [4] consolidated the theory of B-trees and their variants, giving the data structure community a canonical reference. This survey described the B-tree, B⁺-tree, and B*-tree (Knuth, 1973) in a unified framework and analyzed their performance characteristics in the disk model. Comer's article cemented the B⁺-tree as the preferred variant for database index structures—a status it retains today.

3.3 Database Adoption (1980s)

Through the 1980s, the relational database industry adopted B⁺-trees as the standard index mechanism. IBM's System R prototype—the system that led to SQL—used B-tree indexes for its access paths. Oracle, Ingres, and DB2 all incorporated B⁺-tree indexes in their storage engines. The structure's predictable worst-case performance, space efficiency, and support for range queries made it far superior to hash-based alternatives for the query patterns typical in transactional databases.

During this era, researchers also studied B-tree concurrency control—how to support concurrent reads and writes on a shared B-tree without coarse-grained locking. Lehman and Yao's influential 1981 paper "Efficient Locking for Concurrent Operations on B-Trees" [5] introduced the link-based concurrency protocol (now called the "B-link tree"), which allows concurrent searches and updates without locking entire paths from root to leaf. This work underpins the concurrency mechanisms in modern database systems.

3.4 File System Integration (1990s)

In the 1990s, B-trees migrated from database engines into operating system file systems. Microsoft's NTFS (1993) uses a B-tree variant to store directory entries, enabling efficient file lookup even in directories with hundreds of thousands of files. Apple's HFS+ (1998) and its successor APFS (2017) similarly use B-tree structures for volume metadata.

The Linux kernel's ext3 and ext4 filesystems use HTree (hash-tree) directory indexing, a variant of B-trees where keys are hashes of filenames. The Btrfs filesystem (2009), now a standard option in major Linux distributions, uses a copy-on-write B-tree as its primary on-disk structure, enabling features such as snapshots, checksumming, and online defragmentation [6].

3.5 SSD and NVM Challenges (2000s–Present)

The widespread deployment of solid-state drives (SSDs) in the 2000s forced a reexamination of B-tree design. Unlike spinning disks, SSDs have no seek latency, exhibit asymmetric read/write performance, and suffer write amplification effects when pages are modified in place (because SSDs must erase entire blocks before writing). These characteristics can make the frequent small writes produced by B-tree insertions—including page splits—particularly costly on SSD storage.

Researchers responded by developing write-optimized variants. The Log-Structured Merge-tree (LSM-tree), introduced by O'Neil et al. in 1996 [7] and popularized by Google's Bigtable and LevelDB, batches writes in memory and flushes them sequentially, avoiding the random write patterns of B-trees. However, LSM-trees sacrifice read performance and require periodic compaction. The B ϵ -tree (Brodal and Fagerberg, 2003; Bender et al., 2007) [8] takes a middle road: it is a B-tree variant that buffers updates in internal nodes, achieving write-optimal I/O complexity while maintaining good read performance.

4. Recent Advancements

4.1 The B ϵ -Tree and Write-Optimized Structures

The B ϵ -tree (pronounced "B-epsilon tree") was introduced by Brodal and Fagerberg (2003) and further developed by Bender et al. (2007) [8]. In a B ϵ -tree, each internal node of a B-tree with branching factor B has a buffer of size B ϵ messages (for $0 < \epsilon < 1$). Insertions are written to the root's buffer; when a buffer fills, its messages are flushed to the next level. This cascading mechanism achieves $O((1/B^{\{1-\epsilon\}}) \log_B N)$ amortized I/Os per insert—an exponential improvement over standard B-trees—while searches still cost $O(\log_B N + \log N / B^\epsilon)$. TokuDB (now Percona TokuDB) shipped a production implementation of the B ϵ -tree, achieving dramatic write throughput improvements on benchmark workloads.

4.2 Cache-Oblivious B-Trees

Classical B-trees require the block size B to be known at design time in order to size nodes appropriately. In 2000, Frigo, Leiserson, Prokop, and Ramachandran introduced the cache-oblivious model [9], in which algorithms achieve near-optimal I/O performance on any memory hierarchy (cache, TLB, RAM, disk) without being tuned to specific hardware parameters. The cache-oblivious B-tree (Bender et al., 2000) uses a van Emde Boas layout to arrange tree nodes in memory so that subtrees of any size occupy contiguous memory blocks, achieving $O(\log_B N)$ cache misses per operation without knowing B.

4.3 In-Memory B-Trees and CPU Cache Optimization

As RAM capacities grew to accommodate many workloads entirely in memory, researchers refocused on CPU cache performance rather than disk I/O. Kim et al. (2010) introduced the CSB⁺-tree (Cache-Sensitive B⁺-tree) [10], which stores all children of a node in a single contiguous array, eliminating the child-pointer array. This design reduces the node size so that more keys fit in a single cache line, reducing cache misses for in-memory workloads. Experiments showed 2–3 $\times$ throughput improvements over pointer-based B⁺-trees on integer keys.

4.4 Learned Indexes

A recent and provocative line of work challenges whether index structures need to be trees at all. Kraska et al. (2018) introduced the Recursive Model Index (RMI) [11], which replaces B-tree nodes with learned models (linear regression, neural networks) that predict the position of a key in a sorted array. For read-heavy workloads on static datasets, learned indexes can be 2–100 $\times$ faster and use 10–100 $\times$ less memory than B-trees. However, they handle insertions and deletions poorly. The ALEX index (Ding et al., 2020) [12] extends learned indexes to support updates using a gapped array and a hierarchy of learned models, closing much of the gap with B-trees on write-heavy workloads.

4.5 B-Trees in Modern Databases

Despite decades of alternatives, the B⁺-tree remains the dominant index structure in production relational databases. PostgreSQL's default index type is the B-tree, implemented in *src/backend/access/nbtree/*, and supports all comparison operators [13]. MySQL's InnoDB engine uses B⁺-trees for both the clustered primary-key index (where leaf pages store full row data) and secondary indexes. SQLite uses a B-tree variant as its entire storage format, with one B-tree per table and one per index, all packed into a single file. MongoDB's WiredTiger engine also uses B-trees as its primary storage structure. These production deployments demonstrate that B-trees' combination of predictable performance, simplicity, and range-query support continues to outweigh the advantages of write-optimized or learned alternatives for general-purpose OLTP workloads.

5. Implementation

5.1 Design Overview

We implemented a B-tree of configurable minimum degree t in standard C++17 (*btree.cpp*). The implementation follows the pseudocode in Cormen et al. CLRS Chapter 18 [14], adapted to use idiomatic C++ (`std::vector` for dynamic key/child arrays). The tree is a classic B-tree (not a B⁺-tree), meaning data is stored at every node rather than exclusively at leaves.

5.2 Data Structures

The implementation consists of two classes:

- BTreeNode: node-level operations

```
struct BTreeNode {  
    int t;           // minimum degree  
    std::vector<int> keys;  
    std::vector<BTreeNode*> children;  
    bool leaf;  
    // ... member functions  
};
```

- BTree: public API wrapper

5.3 Key Algorithms

Search

Search is straightforward: at each node, perform a linear scan through the sorted keys to find either the matching key (return success) or the appropriate child pointer to recurse into. Because nodes store at most $2t-1$ keys and the tree height is $O(\log_t N)$, the total work is $O(t \log_t N)$.

Insertion

Insertion proceeds via a *proactive split* strategy: as we descend from the root toward the insertion point, any full node (one with $2t-1$ keys) is split before recursing into it. This guarantees that whenever we need to add a key to a node's parent, the parent has room. The split operation takes the median key of the full node, promotes it to the parent, and distributes the remaining keys into two nodes of size $t-1$. If the root itself is full, a new root is created, increasing the tree's height by one.

Deletion

Deletion is the most complex operation, handling three cases:

1. If the key is in a leaf node, remove it directly.
2. If the key is in an internal node, replace it with its in-order predecessor or successor (from a child with $\geq t$ keys), then recursively delete from that child.
3. If the relevant child has only $t-1$ keys (the minimum), first restore it to at least t keys by borrowing from a sibling or merging with a sibling and the separator key from the parent, then proceed with deletion.

5.4 Memory Management

Each `BTreeNode` is heap-allocated. The `BTree` destructor recursively deletes the tree. During merge operations, care is taken to clear the merged child's children vector before deletion to prevent double-free errors.

5.5 Source Code

The full source is included in the submission zip file as `btree.cpp`. The file contains both the implementation and a `main()` function that runs correctness tests and timing benchmarks.

6. Experiments and Evaluation

6.1 Correctness Tests

I ran five automated correctness tests:

- Sorted order after inserting a small set of arbitrary keys.
- Search returns true for present keys and false for absent keys.
- Deleting a leaf key removes it without disturbing other keys.
- Deleting an internal key maintains sorted order throughout the tree.
- Large random insertion (1000 keys): the in-order traversal matches a sorted copy of the inserted values.

All five tests pass:

```
=== Correctness Tests ===  
[PASS] Sorted order after insertion  
[PASS] Search correctness  
[PASS] Delete leaf key  
[PASS] Delete internal key  
[PASS] Large random insertion invariant
```

6.2 Performance Benchmarks

I benchmarked insertion and search over $N \in \{10,000; 50,000; 100,000\}$ random integer keys for tree orders $t \in \{2, 5, 10, 50, 100\}$. Times are wall-clock milliseconds measured with `std::chrono::high_resolution_clock` on a modern laptop. Height is the depth of the deepest leaf.

t	N	Insert (ms)	Search (ms)	Height
2	10,000	3.32	0.91	10
2	50,000	31.75	4.27	12
2	100,000	83.54	10.38	12
5	10,000	1.14	0.87	4
5	50,000	7.67	6.06	5
5	100,000	27.18	15.21	5
10	10,000	0.83	0.71	3
10	50,000	5.27	4.75	3
10	100,000	12.43	12.66	4
50	10,000	0.74	0.79	2
50	50,000	4.57	4.60	2
50	100,000	9.89	19.15	2
100	10,000	0.75	0.81	1

100	50,000	4.94	5.68	2
100	100,000	11.00	11.99	2

Table 2: Benchmark results. Times are in milliseconds. Height is depth of deepest leaf.

6.3 Analysis

The results confirm several theoretical predictions. First, tree height decreases dramatically as t increases: for $N = 100,000$, height drops from 12 ($t = 2$) to 2 ($t = 50$), consistent with the formula $h \leq \log_t[(N+1)/2]$. In a disk-based system, this height reduction directly translates to a $6\times$ reduction in I/O operations per lookup.

Second, insertion is fastest for moderate to large t values (10–50), where the combination of low height and manageable node-level linear scans minimizes total work. Very large t (e.g., 100) shows slightly worse search times at large N because scanning 199 keys per node linearly becomes costly in memory—a real system would use binary search within nodes.

Third, the $t = 2$ case (a 2–3 tree, the minimum B-tree) is consistently the slowest due to its large height, illustrating why real systems choose t in the hundreds to thousands to match disk block sizes.

7. Discussion

7.1 Trade-offs

The central trade-off in B-tree design is between node size and operation cost. Larger nodes (higher t) reduce height and I/O count but increase the cost of linear key search within nodes and of splitting/merging (which must copy more data). In practice, the optimal t is determined by the block size: set node size = block size, then $t \approx \text{block_size} / (2 \times \text{key_size})$.

B-trees also trade write complexity for read performance. Every insertion may trigger a cascade of splits; every deletion may trigger borrows or merges. These operations are $O(h)$ in the worst case and may require multiple dirty pages to be flushed to disk, increasing write amplification. Write-optimized structures like LSM-trees sacrifice some read performance to reduce this write pressure.

7.2 Limitations of Our Implementation

Our implementation is a proof-of-concept and differs from production B-trees in several important ways:

- Keys are 32-bit integers; production indexes support variable-length byte strings.
- No disk serialization: nodes live in heap memory rather than mapped disk blocks.
- Node scans are linear; binary search within nodes would improve performance for large t .
- No concurrency control: the implementation is single-threaded.
- No transaction support or crash recovery (WAL/redo logging).

7.3 What Was Learned

Implementing the B-tree deletion algorithm in particular revealed the subtlety of maintaining the minimum-degree invariant. The three cases and their sub-cases (borrow from left sibling, borrow from right sibling, merge) must be handled carefully to avoid leaving nodes underfull. The proactive split strategy for insertion, by contrast, is elegantly simple and ensures that no parent is ever full when a child needs to push a median key up.

The benchmarks provided concrete intuition for a fundamental lesson of systems design: the optimal data structure depends on the hardware. On disk, $t = 512$ (a common database page size choice) is dramatically better than $t = 2$. But even in memory, moderate t (around 10–50) wins because it reduces both pointer chasing and cache pressure compared to a binary tree.

8. Conclusion and Future Work

B-trees have remained one of the most important data structures in computer science for over fifty years. From their invention by Bayer and McCreight in 1972 through their central role in modern relational databases, file systems, and key-value stores, they demonstrate how a clean theoretical insight—match the tree's branching factor to the hardware's block size—can yield a structure with outstanding practical performance.

This report traced the B-tree's history from its origins through the B⁺-tree, file-system adoption, the challenges posed by SSDs, and recent advances including write-optimized B_ε-trees, cache-oblivious variants, and learned indexes. Our C++ implementation and benchmarks confirmed the key theoretical result: increasing the minimum degree t reduces tree height logarithmically, directly reducing I/O cost in disk-based systems.

Directions for future work include:

- Extending the implementation to B⁺-trees with a linked leaf list supporting efficient range queries.
- Adding disk serialization to benchmark real I/O performance against simulated block sizes.
- Implementing binary search within nodes and comparing against the current linear scan.
- Adding concurrent access support using a lock-coupling or link-based (B-link) protocol.
- Comparing the B-tree empirically against an LSM-tree (e.g., LevelDB) on write-heavy workloads.

References

- [1] A. Aggarwal and J. S. Vitter, "The input/output complexity of sorting and related problems," *Communications of the ACM*, vol. 31, no. 9, pp. 1116–1127, 1988.
- [2] R. Bayer and E. McCreight, "Organization and maintenance of large ordered indexes," *Acta Informatica*, vol. 1, pp. 173–189, 1972.
- [3] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [4] D. Comer, "The ubiquitous B-tree," *ACM Computing Surveys*, vol. 11, no. 2, pp. 121–137, Jun. 1979.
- [5] P. L. Lehman and S. B. Yao, "Efficient locking for concurrent operations on B-trees," *ACM Transactions on Database Systems*, vol. 6, no. 4, pp. 650–670, 1981.
- [6] O. Rodeh, J. Bacik, and C. Mason, "BTRFS: The Linux B-tree filesystem," *ACM Transactions on Storage*, vol. 9, no. 3, pp. 1–32, 2013.
- [7] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, pp. 351–385, 1996.